

# Kafka

## Production Handbook

Practical Notes for Junior Engineers



From Basics to Real-World  
Production Use Cases



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# Kafka in Production

## 1. What is Apache Kafka?

- Apache Kafka is a distributed platform for **event streaming**.
- It stores, processes and delivers **real time data** at scale.
- It is designed for **high throughput, accuracy and durability**.

## 2. Why does it exist?

- Modern applications generate **continuous streams of data**.
- Traditional systems struggle to handle this at scale.
- Kafka exists to **collect, store and deliver these events reliably** to multiple systems in real time.

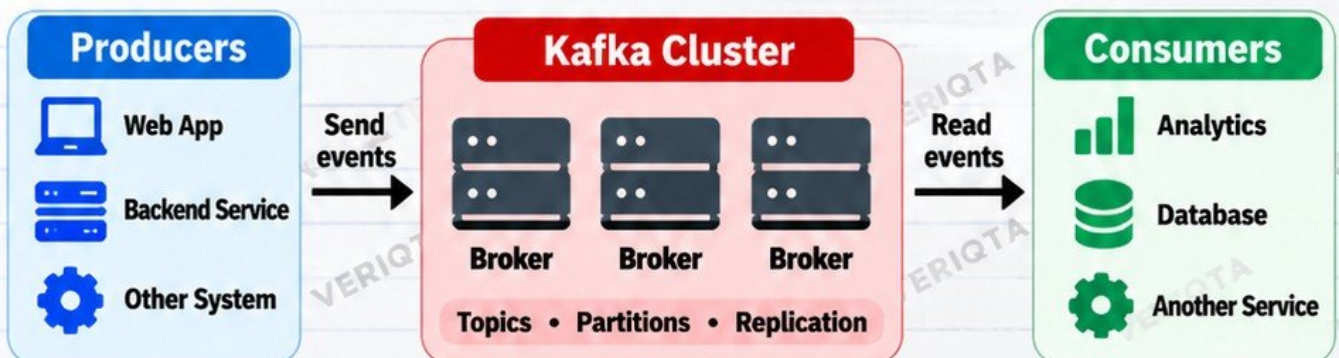
## 3. What problem does it solve?

- Helps **decouple systems**.
- Handles **large volumes** of real time data.
- Delivers data to **multiple consumers**.
- Keeps data available even if a consumer is temporarily down.

## 4. Where is it used?

- Real time data pipelines
- Log and metrics collection
- Microservices communication
- Data integration between systems
- Analytics and machine learning
- Event driven architectures

## 5. Simple Kafka Mental Model



## 6. How it works (High Level)

- 1 Producers send events to a Kafka **topic**.
- 2 Events are stored in **partitions** on brokers.
- 3 Kafka replicates data for **durability**.
- 4 Consumers **read events** from the topic.
- 5 Each consumer tracks its position using **offsets**.



### Junior Engineer Tip

Think of Kafka as a central event highway. Many producers can send data, and many consumers can read the same data independently without affecting each other.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# Kafka Architecture

## 1. Kafka Cluster

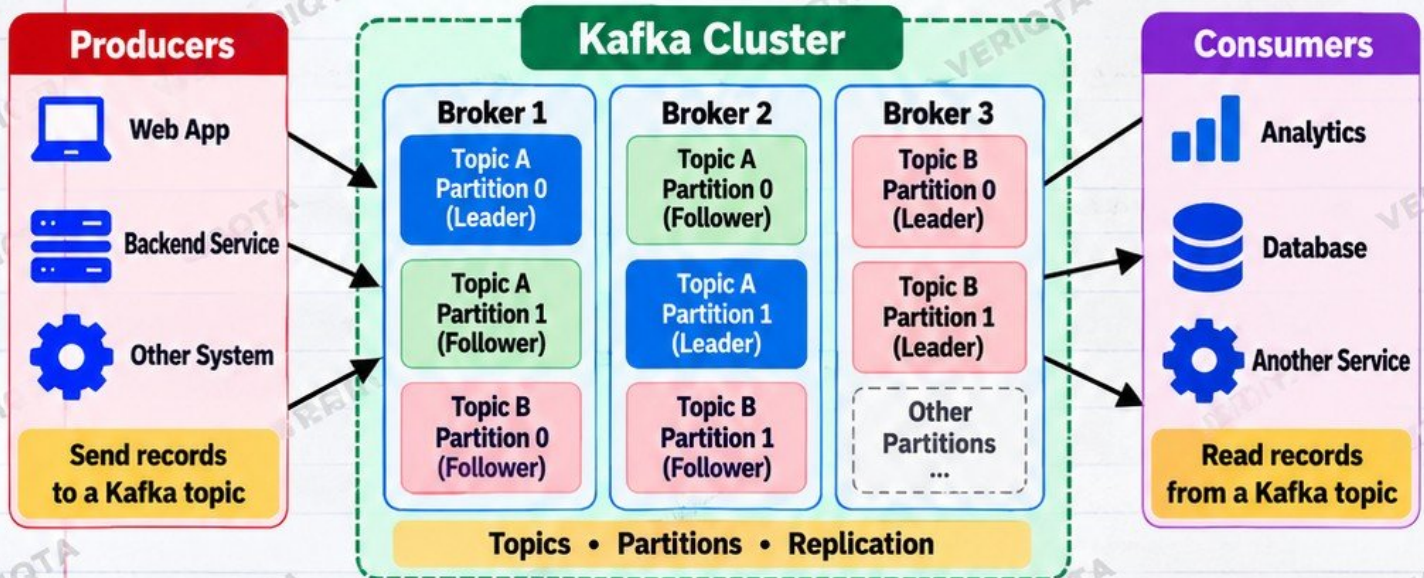
- A Kafka cluster is a **group of brokers** working together.
- It stores and replicates data across multiple servers.
- Provides **high availability, scalability** and **durability**.

## 2. Broker

- A broker is a **Kafka server** that **stores data**.
- Each broker holds a set of **topic partitions**.
- Brokers work together in a **cluster**.

## 3. Topic

- A topic is a **named category** for records (messages).
- Topics are split into **partitions**.
- Example: orders, payments, user-events.



## 4. Partition

- A topic is divided into **partitions**.
- Each partition stores an **ordered sequence** of records.
- Partitions enable **parallel processing**.

## 5. Replication

- Each partition has **multiple replicas** (copies) across brokers.
- One replica is the **leader**.
- Other replicas are **followers**.
- Replication provides **fault tolerance** and **high availability**.

## 6. Producers

- Producers send records to a **Kafka topic**.
- They choose a partition based on a **key** or use **round robin**.
- Producers can send records at **high throughput**.

## 7. Consumers

- Consumers read records from a topic.
- They belong to a **consumer group**.
- Each partition is assigned to **one consumer** in a group at a time.



### Junior Engineer Tip

Think of Kafka as a distributed event log. Producers write events to topics, Kafka stores and replicates them across multiple brokers, and consumers read the events. This decouples systems so they can work independently.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# Topics & Partitions

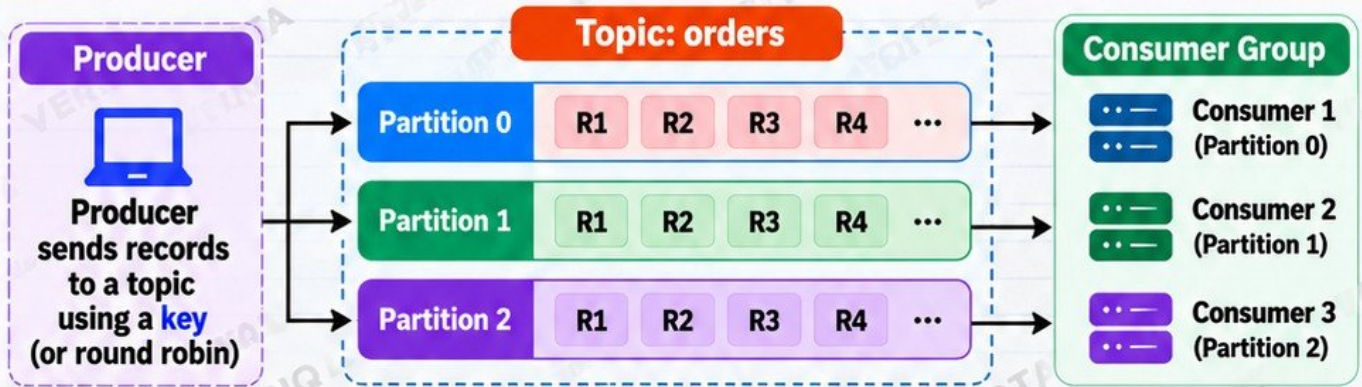
## 1. What is a Topic?

- A topic is a **named category** for **records** (messages).
- Similar to a folder where related events are stored.
- Examples: orders, payments, user-events, logs.

## 2. Why are Topics Partitioned?

- A topic is divided into **multiple partitions**.
- Partitions allow **parallel processing**.
- Each partition stores an **ordered sequence** of records.
- This allows **high throughput** and **scalability**.

### Topic with 3 Partitions (Records are stored in order within each partition)



## 3. Ordering Guarantees

- Records in a partition are **stored in order**.
- Kafka guarantees ordering **only within a partition**.
- There is no global ordering across multiple partitions.

## 4. Partition Keys

- A key decides **which partition** a record goes to.
- Same key always goes to the **same partition**.
- This keeps related records together (for example, all events for one user).
- If no key is provided, Kafka uses **round robin**.

## 5. Why Partition Count Matters

- More partitions add **parallelism** and **higher throughput**.
- Too few partitions can **limit performance**.
- Too many partitions can **increase overhead** and make management harder.
- Choose a partition count based on **current and future load**.

## 6. Example: Sending Records with a Key

```
# Using a key (for example, user-id)
producer.send(
    topic='orders',
    key='user-123',
    value='order created'
)
```

### What happens?

- Records with the same key (user-123) always go to the **same partition**.
- This keeps all events for that user in order.



### Junior Engineer Tip

- Use a meaningful key such as **user-id**, **order-id** or **device-id** when you need to keep related events together.
- Let Kafka choose a partition (**round robin**) only when order between records does not matter.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# Producers & Message Flow

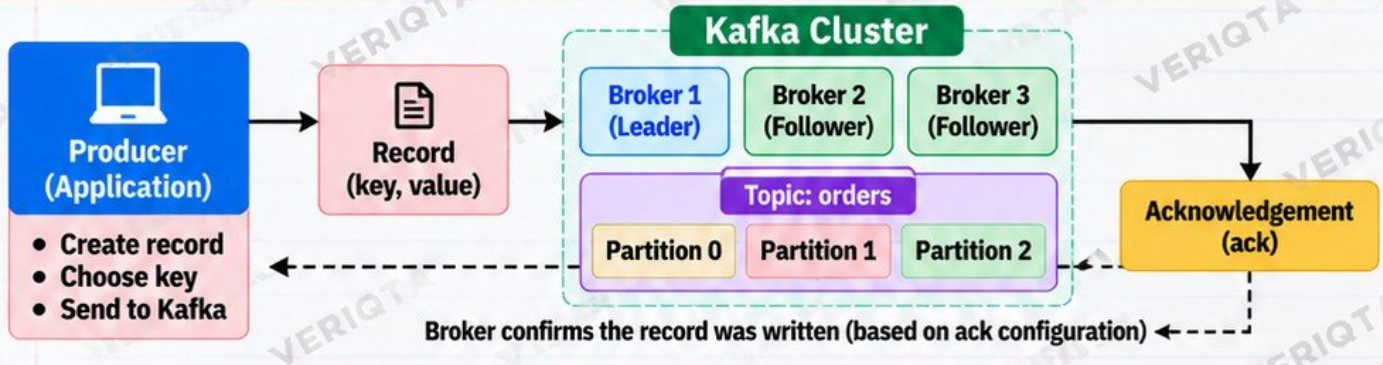
## 1. What is a Producer?

- A producer is an application that **sends records** to a **Kafka topic**.
- Producers can send data from web apps, backend services, IoT devices or any system that generates events.
- A record (message) is a **key, value** and optional **metadata**.

## 2. How a Record is Sent (High Level Flow)

- 1 Producer creates a record (key + value).
- 2 Producer selects a **partition** (based on **key** or using **round robin**).
- 3 Record is sent to the **leader** of the partition.
- 4 Kafka writes the record to its **log** and **replicates** it to follower replicas.
- 5 Broker responds with an **acknowledgement (ack)** to the producer.

## 3. Kafka Producer Message Flow



## 4. Key Selection & Partitioning

- If a **key** is provided, Kafka uses a **hash** of the key to choose a partition.
- Records with the **same key** always go to the **same partition**.
- If no key is provided, Kafka can use **round robin** to distribute records across partitions.
- Use a **meaningful key** (for example, user-id or order-id) when you need related events to stay in order.

## 5. Acknowledgements (acks)

- acks=0** Producer does not wait for a response. (Faster but lower reliability)
- acks=1** Waits for the leader to write the record. (Balanced, commonly used)
- acks=all** Waits for leader and follower replicas. (Higher reliability)

## 6. Batching, Retries & Reliability

- Producers **batch multiple records** to improve throughput.
- If a send fails, the producer can **retry** automatically.
- Enable **idempotence** to avoid duplicate records.
- Retries with **acks=all** provide higher reliability.
- Configure sensible **timeouts** to avoid long delays.

## 7. Example: Sending a Record (Java)

```

ProducerRecord<String, String> record =
    new ProducerRecord<>(
        "orders",
        "user-123",
        "order created"
    );
producer.send(record);
  
```



## Junior Engineer Tip

- Use a **key** when you need all events for the same entity to stay in order.
- Use **acks=all** and enable retries in production.
- Batching improves throughput, especially at high load.
- Monitor producer errors and retry rate.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# Consumers & Consumer Groups

## 1. What is a Consumer?

- A consumer is an application that **reads records** from a Kafka topic.
- Consumers fetch records sequentially from a **partition offset** and process them.
- After processing, the consumer **commits the offset** to track its position.

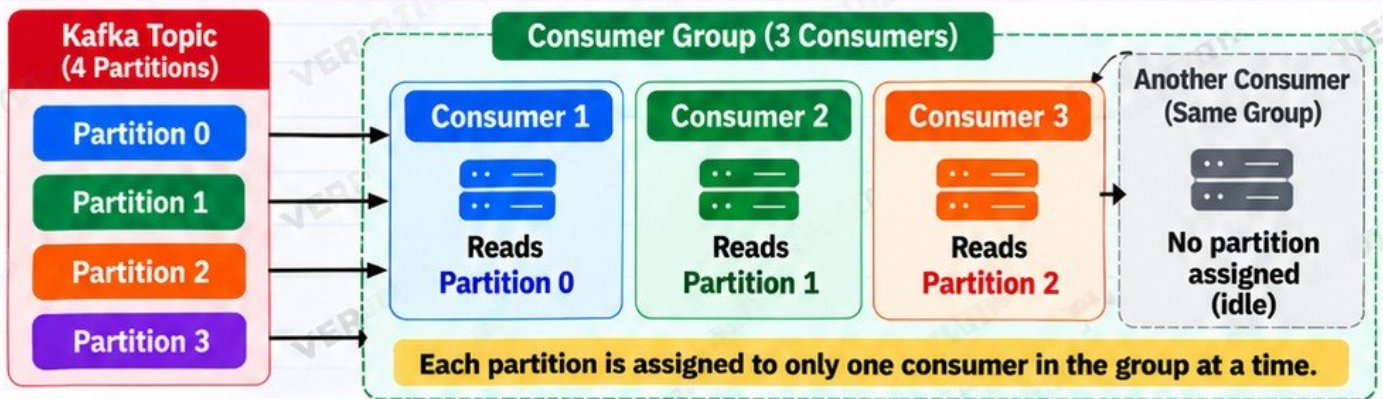
## 2. What is a Consumer Group?

- A consumer group is a **group of consumers** that work together to read from a topic.
- Each partition is assigned to **one consumer** in the group.
- This allows multiple consumers to process different partitions **in parallel**.

## 3. Partition Assignment

- Kafka assigns each partition to **only one consumer** within a consumer group at a time.
- If you have **more partitions** than consumers, some consumers will handle **multiple partitions**.
- If you have **more consumers** than partitions, some consumers will be **idle**.

## 4. How Consumer Groups Work (Example)



## 5. Parallel Processing

- Each consumer in the group processes its assigned partitions **in parallel**.
- This allows **high throughput** and **faster processing**.
- Increase the number of partitions and consumers to **scale processing** (where appropriate).

## 6. Why Only One Consumer per Partition?

- Kafka guarantees **order of records** within a partition.
- Allowing multiple consumers to read the same partition would **break the order guarantee**.
- If you need multiple consumers to process the same data, use **different consumer groups**.

## 7. Real World Example

- An e-commerce system has an **orders topic** with **6 partitions**.
- A consumer group with 3 consumers will process **2 partitions each**.
- This allows **parallel processing** of orders, payments or notifications.

## 8. Example: Consumer Configuration (Java)

```
Properties props = new Properties();
props.put("bootstrap.servers", "kafka:9092");
props.put("group.id", "order-processor");
props.put("key.deserializer", "StringDeserializer");
props.put("value.deserializer", "StringDeserializer");
KafkaConsumer<String, String> consumer =
    new KafkaConsumer<>(props);
consumer.subscribe(Arrays.asList("orders"));
```



### Junior Engineer Tip

- Use a unique **group.id** for each application.
- Do not use the same **group.id** for different applications unless you want them to share the work.
- Use multiple partitions and multiple consumers to increase throughput.
- Monitor consumer lag to ensure your consumers can keep up.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# Offsets & Delivery Semantics

## 1. What are Offsets?

- An offset is a **unique number** that identifies the **position** of a record within a **partition**.
- Offsets **start from 0** and **increase by 1** for each new record.
- Offsets allow consumers to **track** which records they have read.

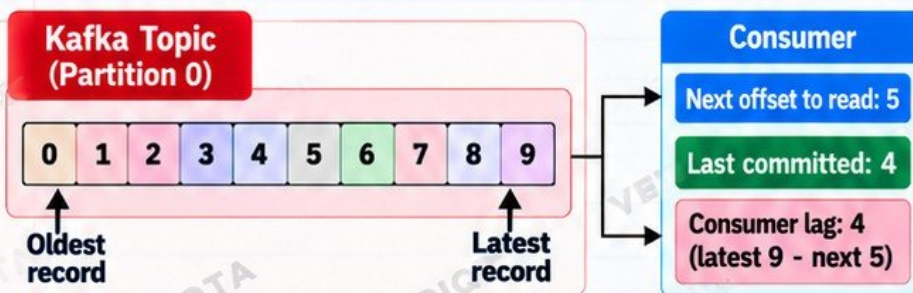
## 2. Committed Offsets

- A committed offset is the **last offset** that a consumer has **successfully processed**.
- Kafka stores **committed offsets** internally (in the `__consumer_offsets` topic).
- When a consumer restarts, it **continues from** the last committed offset.

## 3. Consumer Position

- The consumer position is the **next offset** the consumer will read.
- It can be behind the latest offset (**consumer lag**) or caught up.
- You can **reset** the consumer position to an **earlier or later offset** if needed.

## 4. Offsets, Consumer Position and Commit (Example)



## Junior Engineer Tip

- A consumer reads records from the next offset.
- After processing, it commits the offset.
- If the consumer restarts, it continues from the last committed offset, not the last processed offset.

## 5. At-Most-Once (Lowest Guarantee)

- Records may **be lost** if the consumer processes the record but **fails before committing** the offset.
- Provides **high performance** but **lower reliability**.
- Use case: non-critical or **rebuildable data**.



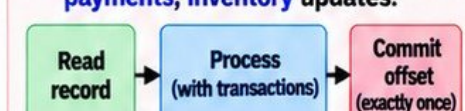
## 6. At-Least-Once (Commonly Used)

- Records are **never lost** but may be processed **more than once**.
- Consumer commits the offset after processing.
- Duplicates can happen if the consumer processes the record but **fails before committing**.
- Use case: **most real world systems**.



## 7. Exactly-Once (Higher Guarantee)

- Records are processed **once and only once** (as seen by the application).
- Requires **idempotent producers, transactions** and proper configuration.
- More complex and may impact **performance**.
- Use case: financial **transactions, payments, inventory updates**.



## 8. Duplicate Processing Risks

- Can happen with **at-least-once** delivery.
- May be caused by consumer crashes, network issues or committing offset after processing.
- Applications must be **idempotent** (safe to process the same message more than once).

## 9. Key Takeaways

- 1 Offset tracks the **position** of a record in a partition.
- 2 Consumers **commit offsets** to remember their progress.
- 3 At-most-once may **lose data**.
- 4 At-least-once may process records **more than once**.
- 5 Exactly-once provides **stronger guarantees** but is more complex.
- 6 Design your application to handle **duplicate records**.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# Replication & High Availability

## 1. What is Replication?

- Replication means storing **multiple copies** of each partition across different brokers.
- It provides **fault tolerance** and **high availability**.
- If one broker fails, another broker can continue to serve the data.

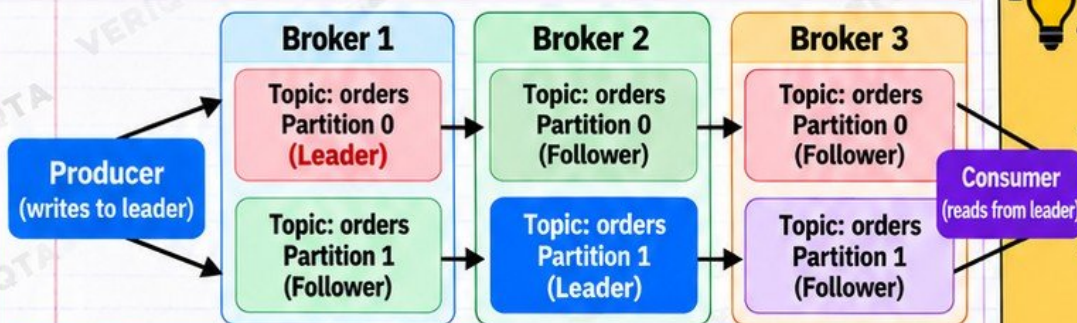
## 2. Leaders and Followers

- Each partition has one **leader** and zero or more **followers**.
- The leader handles all **read and write requests**.
- Followers **replicate** the data from the leader.
- If the leader fails, one follower is **promoted** to become the new leader.

## 3. Replication Factor

- The **replication factor (RF)** determines how many copies of each partition exist.
- Example: RF = 3 means **1 leader + 2 followers**.
- A higher replication factor improves **availability** but uses **more disk space**.
- In production, RF is usually **3 or more**.

## 4. Kafka Partition Replication (Example with RF = 3)



Each partition has 1 leader and 2 followers (RF = 3) stored on different brokers.



### Junior Engineer Tip

- Always use a replication factor of **at least 3** in production.
- Place replicas on different brokers and **different availability zones** if possible.
- This ensures your data remains available even if a broker or a zone fails.

## 5. In-Sync Replicas (ISR)

- The ISR is the list of followers that are **fully caught up** with the leader.
- Only replicas in the ISR are **eligible to become the new leader**.
- If a follower falls behind, it is removed from the ISR until it catches up.

## 6. Leader Election

- If the leader fails, Kafka automatically **promotes one of the in-sync replicas** to become the new leader.
- The election is handled by the **controller (in KRaft mode)**.
- Clients **automatically reconnect** to the new leader.

## 7. Broker Failure Scenario

- 1 A broker fails (for example, Broker 1).
- 2 The leader on that broker is **no longer available**.
- 3 One follower from the ISR is **elected as the new leader**.
- 4 Producers and consumers **automatically connect** to the new leader.
- 5 The partition **remains available** and continues to serve data.

## 8. How Kafka Keeps Partitions Available

- Replication stores **multiple copies** of each partition.
- The ISR ensures only **up-to-date replicas** can become leader.
- Leader election **automatically promotes** a follower if the leader fails.
- Clients reconnect to the new leader **without data loss** (as long as the ISR is maintained).

## 9. Key Takeaways

- ✓ Each partition has **one leader and multiple followers**.
- ✓ Replication provides **fault tolerance** and **high availability**.
- ✓ The ISR ensures **data consistency**.
- ✓ Leader election **keeps partitions available** during failures.
- ✓ Use a replication factor of **3 or more** in production.
- ✓ Keep brokers in **different zones** for higher resilience.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# KRaft & Cluster Metadata

## 1. What is KRaft?

- KRaft (Kafka Raft) is the **metadata management** mode used in modern Kafka.
- It **replaces ZooKeeper** for managing cluster metadata.
- Introduced to **simplify architecture**, reduce operational complexity and improve reliability.

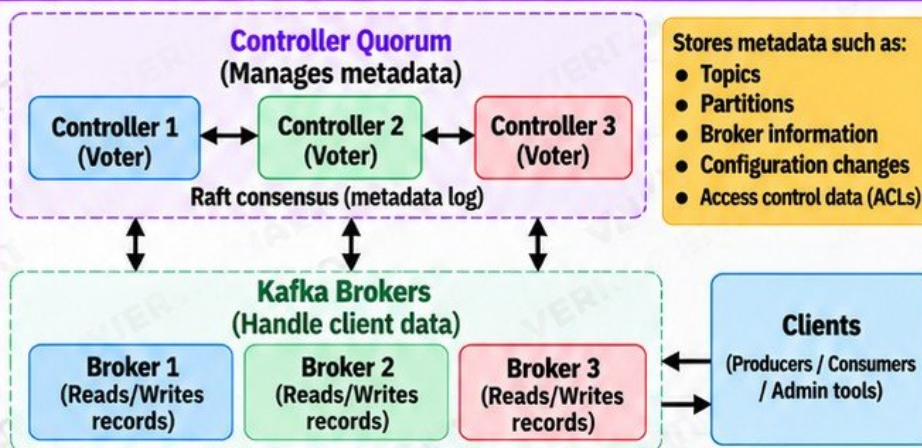
## 2. Why Kafka Uses KRaft Now?

- Removes the **dependency** on **ZooKeeper**.
- **Simplifies** deployment and operations.
- Improves **scalability** and **availability**.
- Uses the **Raft consensus** algorithm for metadata.
- It is the **recommended mode** for new Kafka deployments.

## 3. KRaft vs ZooKeeper

| Feature                    | ZooKeeper (Old)             | KRaft (Current)                        |
|----------------------------|-----------------------------|----------------------------------------|
| Used for                   | Cluster metadata            | Cluster metadata                       |
| Architecture               | Separate ZooKeeper ensemble | Built into Kafka (brokers/controllers) |
| Operational complexity     | Higher                      | Lower                                  |
| Dependency                 | External service required   | No external dependency                 |
| Recommended for new setups | No                          | Yes                                    |

## 4. KRaft Architecture (Simplified)



## 5. Controller Responsibilities

- Manage **cluster metadata**.
- Handle **topic and partition** creation, updates and deletion.
- Manage **broker registration** and **heartbeats**.
- Track the **state of partitions** (leaders, replicas, ISR).
- Handle **configuration changes** in the cluster.
- Coordinate **leader election** for partitions.
- Ensure metadata is **consistently replicated** across the controller quorum.

## 6. Metadata Quorum

- A group of controller nodes that use the **Raft consensus** algorithm.
- Typically requires an **odd number** of controllers (3, 5, etc).
- One controller acts as the **active controller (leader)** and the others are followers.
- The **metadata log** is replicated across the quorum to ensure **consistency** and **fault tolerance**.

## 7. Controller Failover

- 1 If the active controller fails, a **new controller is elected automatically** using Raft.
- 2 The new controller continues managing the **metadata log**.
- 3 Brokers and clients automatically recognize the new controller.
- 4 This ensures **high availability** of cluster metadata.

## 8. Key Benefits of KRaft

- ✓ **No need to manage ZooKeeper.**
- ✓ **Simpler** architecture.
- ✓ Fewer components to operate.
- ✓ Better **scalability**.
- ✓ Improved **fault tolerance**.
- ✓ Faster **metadata operations**.
- ✓ Recommended for new deployments.

## 9. Important Notes

- KRaft is available from **Kafka 2.8+** and is the **default** in newer versions (3.x and above).
- Existing clusters using ZooKeeper can be **migrated** to KRaft.
- Both controllers and brokers run as part of the Kafka cluster (**no separate ZooKeeper**).



### Junior Engineer Tip

- For **new** Kafka clusters, always use KRaft.
- Keep at least **3 controller nodes**.
- Monitor **controller health** and metadata quorum status.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# Kafka Storage Internals

## 1. How Kafka Stores Records

- Kafka stores records in **partition logs** on disk.
- Each topic is divided into partitions.
- Each partition is an **ordered, immutable log** of records.
- Records are written **sequentially** to disk for high performance.
- Kafka does not update or delete individual records.

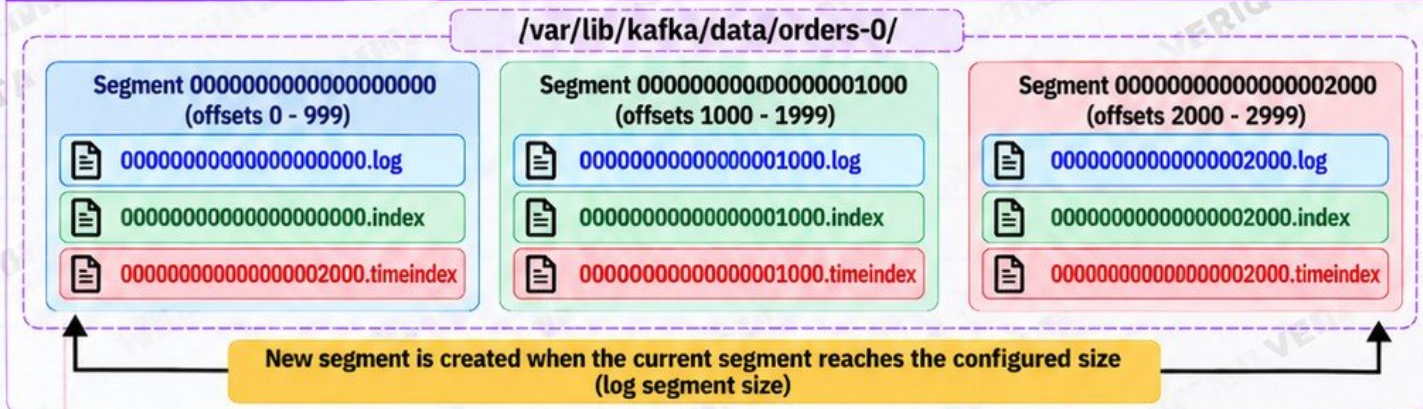
## 2. Partition Log Structure

- Each partition log is split into multiple **segments**.
- A segment is a file that stores a **range of records** (offsets).
- When a segment reaches a configured size, a new segment is created.
- Each segment has an **index file** that helps Kafka quickly find records.

## 3. Files in a Segment

- A segment consists of multiple files:
  - **.log** → actual record data
  - **.index** → maps offsets to file positions
  - **.timeindex** → maps timestamps to file positions
  - **.txnindex** → transaction index (for transactional topics)

## 4. Partition Log on Disk (Example)



## 5. Sequential Writes (High Performance)

- Kafka writes records **sequentially** to the end of the active segment.
- Sequential writes are **very fast and efficient** on disk.
- This design allows Kafka to handle **high throughput** and large amounts of data.

## 6. Retention & Log Cleanup

- Kafka keeps records for a **configured period** of time.
- Deletion can be based on **time (retention.ms)** or **size (retention.bytes)**.
- Two cleanup policies:
  - **Delete** (default) → removes old segments.
  - **Compact** → keeps the latest record for each key.

## 7. Log Cleanup Policies

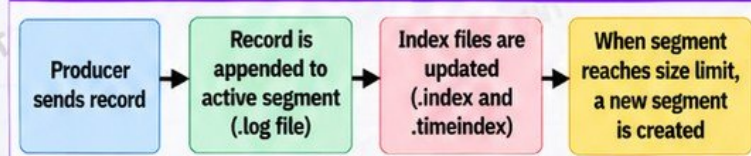
### Delete Policy (Default)

- Old segments are deleted based on time or size.
- Once a segment is deleted, its records are permanently removed.
- Used for most use cases.

### Compact Policy

- Keeps only the latest record for each key.
- Old records with the same key are removed during compaction.
- Useful for stateful data (for example, user profile updates).

## 8. How a Record is Stored (Flow)



## 9. Key Takeaways

- 1 Kafka stores data in **partition logs** on disk.
- 2 Each log is divided into **segments**.
- 3 Kafka writes records **sequentially** for high performance.
- 4 **Retention and cleanup policies** control how long data is kept.
- 5 **Log compaction** keeps the latest record for each key.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# Retention & Log Compaction

## 1. What is Retention?

- Retention controls **how long** Kafka keeps records in a topic.
- Records are **automatically deleted** after a time period or when the topic reaches a size limit.
- It helps manage **disk space** and control data lifecycle.

## 2. Types of Retention

- Time-based retention:** deletes records after a configured time (e.g. 7 days).
- Size-based retention:** deletes old segments when the topic reaches a configured size.
- You can configure **one or both**.

## 3. Retention Configuration

# Time-based retention (default: 7 days)  
`retention.ms=604800000`

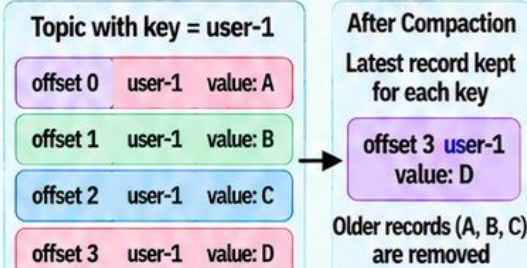
# Size-based retention (default: unlimited)  
`retention.bytes=10737418240` # 10 GB

# Both can be used together  
`retention.ms=604800000`  
`retention.bytes=10737418240`

## 4. What is Log Compaction?

- Log compaction keeps the **latest record for each key** and removes older records with the same key.
- It **does not delete** records based on time.
- It is useful for keeping the **latest state** in a topic.

## 5. How Log Compaction Works



## 6. Tombstones

- A tombstone is a record with a **null value** for a key.
- It marks a **key for deletion** during log compaction.
- After compaction, all records for that key are removed.
- Tombstones are kept for a **limited time** (`delete.retention.ms`), before being removed.

## 7. Retention vs Log Compaction (Comparison)

| Feature            | Time/Size-Based Retention                 | Log Compaction                                  |
|--------------------|-------------------------------------------|-------------------------------------------------|
| What it does       | Deletes old records based on time or size | Keeps only the latest record for each key       |
| Based on           | Time or topic size                        | Key (not time)                                  |
| Use case           | Logs, metrics, events                     | Application state, configuration, user profiles |
| Deletes records    | Yes (old records)                         | Yes (older records with same key)               |
| Keeps latest value | No (deletes by time/size)                 | Yes (keeps latest per key)                      |

## 8. Common Use Cases

| Time/Size-Based Retention                                                                                                                                                             | Log Compaction                                                                                                                                                                 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>Application logs</li> <li>Metrics and monitoring data</li> <li>Clickstream events</li> <li>Audit logs</li> <li>Temporary event data</li> </ul> | <ul style="list-style-type: none"> <li>User profiles</li> <li>Configuration data</li> <li>Product catalog</li> <li>Feature flags</li> <li>Latest state of an entity</li> </ul> |



### Junior Engineer Tip

- Use time-based retention for event data.
- Use log compaction for data where you only need the latest state.
- For tombstones, set an appropriate `delete.retention.ms` (for example, 24 hours).

## 9. Risks and Misconfiguration

- Setting `retention.ms` too low can delete data before consumers read it.
- Setting `retention.bytes` too low can cause unexpected data loss.
- Misunderstanding log compaction may lead to assuming **all history** is available (only the latest value per key is kept).
- Tombstones removed too early** can lead to "resurrected" keys if old records still exist.
- Not monitoring **disk usage** can cause the broker to run out of space.

## 10. Key Takeaways

- Retention controls how **long** Kafka keeps records.
- You can configure **time-based and size-based** retention.
- Log compaction keeps only the **latest record** for each key.
- Tombstones are used to **delete keys** during compaction.
- Choose the right **cleanup policy** based on your use case.
- Misconfiguration can lead to unexpected data loss or missing data.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# Production Configuration

## 1. Replication Factor (RF)

- Determines how **many copies** of each partition are stored across different brokers.
- Typical **production value** is **3 or more**.
- Higher **RF improves availability and fault tolerance** but uses more disk space.
- RF should be at least **3** in production.

## 2. min.insync.replicas (ISR)

- Defines the **minimum number** of replicas that **must acknowledge a write** for it to be considered successful.
- Prevents **data loss** when a leader is isolated.
- Typical value: **2** (when RF = 3).
- If too low, you risk **data loss**. If too high, you may have **availability issues**.

## 3. Producer Acks

- Controls when **the producer** considers a record successfully written.
- **acks=0** → highest performance but no guarantee.
- **acks=1** → waits for **leader only**.
- **acks=all (or -1)** → waits for all in-sync replicas (recommended for production).

## 4. Producer Retries

- Allows the producer to **automatically retry failed** writes.
- Use a **high retry value** (e.g. Integer.MAX\_VALUE) in production.
- Retries help handle **temporary network issues** and leader elections.

## 5. Idempotence

- Ensures the producer does not **send duplicate records** even if retries happen.
- Enable **idempotence** in production (`enable.idempotence=true`).
- Works with **acks=all** and **retries** to provide stronger delivery guarantees.
- **Recommended** for most production workloads.

## 6. Retention Configuration

- Controls how **long** records are **kept** in a topic.
- Retention can be **time-based** (`retention.ms`) or **size-based** (`retention.bytes`).
- Set based on **business and compliance requirements**.
- Ensure **disk space** is monitored to avoid fill-up.

## 7. Partition Count

- Determines the level of **parallelism** and **throughput** for the topic.
- More partitions allow **higher throughput** and more **consumers**.
- Choose a partition count based on expected **traffic and future growth**.
- Increasing **partitions** later is possible but can be **disruptive**, so plan ahead.

### When to Increase Partitions?

- High message throughput
- More consumers needed
- Scaling for future growth
- Workload is evenly distributed by key
- Use monitoring data to decide



### Junior Engineer Tip

- Start with a reasonable partition count based on expected traffic.
- You can increase partitions later if needed.
- Do not create too many partitions, as it increases metadata and management overhead.

## 8. Recommended Production Settings (Example)

| Setting             | Recommended Value                      | Why                                   |
|---------------------|----------------------------------------|---------------------------------------|
| replication.factor  | 3                                      | High availability and fault tolerance |
| min.insync.replicas | 2                                      | Prevents data loss (with RF = 3)      |
| acks (producer)     | all (-1)                               | Waits for all in-sync replicas        |
| retries (producer)  | Integer.MAX_VALUE                      | Handles temporary failures            |
| enable.idempotence  | true                                   | Prevents duplicate records            |
| retention.ms        | Set based on use case (e.g. 7 days)    | Keeps data for required period        |
| partition count     | Based on traffic (e.g. 6, 12, 24, ...) | Allows parallelism and future growth  |

## 9. Key Takeaways

- 1 Use RF = 3 and min.insync.replicas = 2 for better reliability.
- 2 Use **acks=all**, **high retries** and **enable idempotence** for safe production writes.
- 3 Configure **retention** based on business requirements.
- 4 Choose **partition count** based on expected traffic and growth.
- 5 Monitor **cluster health**, **ISR** and **disk usage**.
- 6 **Test** configuration changes in a non-production environment first.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# Kafka Networking

## 1. What is Kafka Networking?

- Kafka uses **TCP networking** for all communication between clients and brokers.
- Clients connect to a broker using a **bootstrap server**.
- After the initial connection, the client uses **metadata** to connect to the correct broker for each partition.
- Correct network configuration is essential for **reliable communication**.

## 2. Listeners and Advertised Listeners

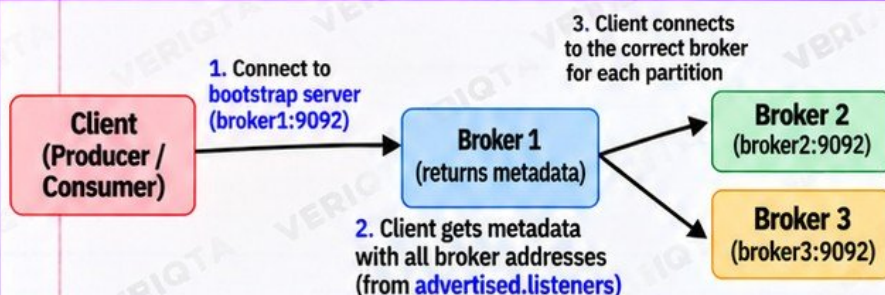
- listeners**: Defines the addresses and ports the broker listens on inside the network.
- advertised.listeners**: Tells clients the host and port to use when connecting to this broker.
- They can be **different**, especially in container or cloud environments.
- If **advertised.listeners** is **wrong**, clients can connect to the **bootstrap server** but **fail to connect** to other brokers.

## 3. Example Configuration

```
# Broker configuration
listeners=PLAINTEXT://0.0.0.0:9092,
INTERNAL://0.0.0.0:9093
advertised.listeners=PLAINTEXT://
broker1.example.com:9092,
INTERNAL://broker1:9093
listener.security.protocol.map=
PLAINTEXT:PLAINTEXT,INTERNAL:PLAINTEXT
```

- listeners** → used by the broker internally.
- advertised.listeners** → the address clients use to connect.

## 4. Client to Broker Connection Flow



## 5. Common Ports

| Port            | Usage                                      |
|-----------------|--------------------------------------------|
| 9092            | Common Kafka port (PLAINTEXT)              |
| 9093            | Often used for internal communication      |
| 9094            | Often used for SSL/TLS (secure connection) |
| Any custom port | Used based on your network setup           |

## 6. Network Environments

| Local / Bare Metal                                                                                                                                        | Docker / Containers                                                                                                                                                                                                            | Cloud (AWS, Azure, GCP)                                                                                                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>Use host IP or hostname.</li> <li>Ensure the client can reach the broker on the <b>advertised port</b>.</li> </ul> | <ul style="list-style-type: none"> <li><b>listeners</b> binds to 0.0.0.0 inside container.</li> <li><b>advertised.listeners</b> should use the host IP or hostname that clients can access (outside the container).</li> </ul> | <ul style="list-style-type: none"> <li>Use public or private DNS names.</li> <li>Ensure security groups and firewall allow the Kafka ports.</li> <li>Use load balancers if needed, with correct <b>advertised.listeners</b>.</li> </ul> |

## 7. The Classic Problem

- "Bootstrap works but broker connection fails"**
- Client can connect to the **bootstrap server**.
  - After getting **metadata**, it tries to connect to other brokers using the addresses from **advertised.listeners**.
  - If those addresses are **not reachable** (e.g. internal container names), the connection fails.
  - This is common in **Docker, Kubernetes and cloud setups**.

## 8. Troubleshooting Steps

- Check your **advertised.listeners** configuration.
- Verify the client can **reach each broker address and port** (from the client machine).
- Test using **telnet** or **nc** (e.g. telnet broker2:9092).
- Check **DNS resolution** (use the correct hostname or IP).
- Ensure **firewall** and **security groups** allow the ports.
- Look at broker logs for connection errors.

## 9. Common Issues

- Wrong **advertised.listeners** (configuration uses internal container name).
- Port not open in **firewall or security group**.
- Incorrect **DNS or hostname**.
- Client and broker in **different networks** without proper routing.
- Load balancer not configured correctly.



## Junior Engineer Tip

- Always set **advertised.listeners** to an address that clients can **actually reach**.
- In Docker, use your host IP or domain name, not the container name.
- Test all broker addresses from the client environment.
- If bootstrap works but brokers fail, **99%** of the time it is an **advertised.listeners** or network issue.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# Kafka Security

## 1. Why Kafka Security Matters?

- Kafka often carries **sensitive data** (user data, payment events, business data).
- Without security, anyone who can reach the cluster can **read, write or delete** data.
- A compromised Kafka cluster can lead to **data leaks**, data tampering and **service disruption**.
- **Never expose Kafka directly to the public internet.**

## 2. Security Layers in Kafka



**Authentication**  
Who are you?  
(Identity)



**Authorization**  
What are you allowed to do?  
(Permissions / ACLs)



**Encryption**  
How is the data protected?  
(TLS)



**Auditing**  
Track and monitor access and operations.

## 3. TLS Encryption

- TLS encrypts **data in transit** between clients and brokers.
- Prevents **eavesdropping** and **data tampering**.
- Uses **SSL/TLS certificates** (CA, server and client certificates).
- Should be enabled for both internal and external communication.
- In production, always use **strong TLS settings (TLS 1.2+)**.

## 4. SASL Authentication

- SASL is used to **authenticate** clients to Kafka.
- Common mechanisms:
  - **SASL/PLAIN** (username/password)
  - **SASL/SCRAM** (username/password with hashing)
  - **SASL/OAUTHBEARER** (for cloud and modern environments)
  - **SASL/GSSAPI** (Kerberos, used in enterprise environments)

## 5. How Secure Client Connection Works



## 6. Authorization with ACLs

- ACLs (Access Control Lists) control what authenticated users can do.
- Permissions can be set on:
  - **Topics** (read, write, describe)
  - **Consumer groups** (read, describe)
  - **Cluster level operations** (create topics, delete topics)
  - **Transactional IDs**
- Use the principle of **least privilege** (only allow what is required).

## 7. Example ACLs

| User / Application | Resource      | Permission      |
|--------------------|---------------|-----------------|
| app-producer       | topic: orders | Write, Describe |
| app-consumer       | topic: orders | Read, Describe  |
| monitoring-user    | cluster       | Describe        |
| admin-user         | cluster       | All (Admin)     |

## 8. Credential Management

- Store credentials **securely** (e.g. secret management tools).
- Use **strong passwords** or SASL mechanisms like SCRAM or OAuth.
- **Rotate credentials regularly.**
- **Avoid hardcoding** credentials in applications or scripts.

## 9. Best Practices

- ✓ Enable TLS for all environments.
- ✓ Use SASL for authentication.
- ✓ Configure ACLs with least privilege.
- ✓ Do not expose Kafka to the **public internet**.
- ✓ Use **firewall, VPN** or private network.
- ✓ Monitor and **audit access logs**.
- ✓ Keep certificates and credentials secure and rotate them.
- ✓ Regularly review and update ACLs.

## 10. Risks of Poor Security

- ✗ Unauthorised users can read sensitive data.
- ✗ Attackers can produce fake or malicious messages.
- ✗ Data can be deleted or modified.
- ✗ Cluster can be used for abuse (spam, DDoS).
- ✗ Compliance and legal issues (GDPR, HIPAA, etc).

## 11. Key Takeaways

- 1 Use **TLS** to encrypt data in transit.
- 2 Use **SASL** for authentication.
- 3 Use **ACLs** for authorization.
- 4 Follow **least privilege**.
- 5 Do not expose Kafka **publicly**.
- 6 Monitor, **audit** and **rotate** credentials.

## Junior Engineer Tip

- In most production environments, Kafka is placed in a **private network** and accessed through **secure connections** (VPN, VPC, or peering).
- **Never open Kafka ports** (like 9092) to the **public internet**.
- Always follow your **organisation's security and compliance requirements**.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# Monitoring Kafka

## 1. Why Monitor Kafka?

- Ensures the cluster is **healthy** and running as expected.
- Helps **detect issues early** before they impact producers, consumers or the business.
- Identifies **performance bottlenecks**, capacity issues and failures.
- Supports **reliability, availability and data durability**.
- Provides visibility into both **infrastructure** and **application** behavior.

## 2. Key Things to Monitor

-  **Broker health** (up / down, resource usage)
-  **Partition health** (under-replicated, offline)
-  **Request latency** (produce and consume)
-  **Throughput** (messages per second)
-  **Disk usage** (log directory, disk space)
-  **JVM health** (heap, GC, memory, threads)

## 3. Important Kafka Metrics

| Metric                    | What It Tells You                                   |
|---------------------------|-----------------------------------------------------|
| BrokerCount               | Number of active brokers                            |
| UnderReplicatedPartitions | Partitions not fully replicated (risk of data loss) |
| OfflinePartitionsCount    | Partitions not available (unreachable)              |
| RequestLatency            | Time taken to process produce/consume requests      |
| MessagesInPerSec          | Throughput for producers                            |
| BytesInPerSec             | Amount of data produced                             |
| BytesOutPerSec            | Amount of data consumed                             |
| DiskUsage                 | Used vs available disk space                        |
| JVMHeapUsage              | Memory usage by Kafka                               |
| GCCount / GCTime          | Garbage collection activity                         |
| PurgatorySize             | Waiting requests (high = issue)                     |
| ActiveControllerCount     | Should be 1 (in KRaft)                              |

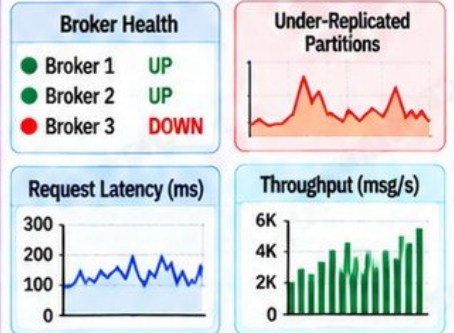
## 4. Partition Health

-  **Healthy**  
All replicas are in-sync (ISR).
-  **Under-Replicated**  
One or more replicas are not in-sync (at risk).
-  **Offline**  
Partition is not available (unreachable).

## 5. Infrastructure vs Application Signals

- | Infrastructure Signals (Cluster Health)                                                                                                                                                                                                                                                              | Application Signals (Client Behavior)                                                                                                                                                                                                                                                                                |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>Broker up or down</li> <li>CPU and memory usage</li> <li>Disk usage and I/O</li> <li>Network throughput</li> <li>Under-replicated partitions</li> <li>Offline partitions</li> <li>Controller health (in KRaft)</li> <li>JVM and garbage collection</li> </ul> | <ul style="list-style-type: none"> <li>Producer request latency</li> <li>Consumer lag</li> <li>Failed produce/consume requests</li> <li>Rebalances frequency</li> <li>Throughput (messages/sec)</li> <li>Application error logs</li> <li>End-to-end processing delay</li> <li>Client retries and failures</li> </ul> |

## 6. Example Monitoring Dashboard



## 7. Disk Usage Monitoring

-  Kafka stores all records on disk, so disk usage is critical.
- Monitor log directory usage.
- Set up alerts when disk usage is high (e.g. > 80%).
- If disk is full, brokers can stop accepting new data.
- Plan for **log retention** and cleanup to manage disk space.

## 8. JVM Health

-  Kafka runs on the **JVM**, so monitor:
  - Heap memory usage** (used vs max)
  - Garbage collection** count and time
  - GC pauses** (should be low)
  - Thread count**
- If the **JVM** is unhealthy, it can cause high latency, slow responses and even broker crashes.

## 9. Key Takeaways

- Monitor broker, partition, client and system metrics.
- Focus on under-replicated and offline partitions.
- Track **request latency** and throughput.
- Monitor **disk usage** and plan for growth.
- Keep an eye on **JVM** health and GC activity.
- Use both **infrastructure** and **application** signals.
- Set up alerts to detect issues early.
- Monitoring is essential for a reliable and highly available Kafka cluster.

## 10. Common Issues and What They Indicate

| Issue                            | Possible Cause                               |
|----------------------------------|----------------------------------------------|
| High under-replicated partitions | Broker failure, network or disk issues       |
| Offline partitions               | Broker down or unreachable                   |
| High request latency             | Broker overloaded, disk slow, GC issues      |
| Low throughput                   | Network issues, insufficient partitions      |
| Disk usage high                  | Retention too long, no cleanup, high traffic |
| Frequent consumer rebalances     | Consumer instability, slow processing        |
| High JVM GC time                 | Memory pressure, misconfigured heap          |



### Junior Engineer Tip

- Always monitor under-replicated and offline partitions.
- Set alerts for disk usage, request latency and JVM GC.
- Use dashboards (e.g. Prometheus + Grafana, or Confluent Control Center).
- Monitor both **infrastructure** and **application** signals.
- Regularly review metrics to ensure cluster health and performance.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



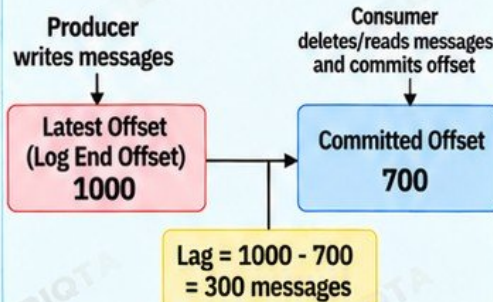
@veriqta

# Consumer Lag

## 1. What is Consumer Lag?

- Consumer lag is the **number of messages** that have been **produced** to a partition but not yet consumed.
- It is the **difference** between the **latest offset** (log end offset) and the consumer's committed offset.
- Lag shows **how far behind** a consumer group is.

## 2. How Consumer Lag Works



## 3. How Lag Develops?

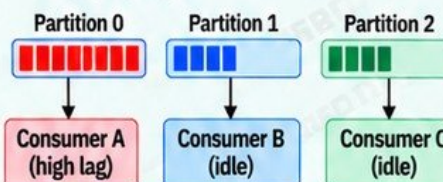
- Consumer processes messages **slower** than they are produced.
- Consumer is **down** or failing.
- Partition imbalance** (some partitions have more data).
- Slow **processing** logic (e.g. database, API calls).
- Insufficient **consumer instances**.
- Network issues or rebalances.

## 4. Identifying Slow Consumers

- Check **consumer lag** for each **consumer group**.
- Identify which **partitions** have the highest lag.
- Look at consumer **processing time** and **commit frequency**.
- Check consumer **logs** for errors or timeouts.
- Monitor **CPU, memory, and I/O** usage of the consumer.

## 5. Partition Imbalance

- Some partitions may receive more data than others.
- This can happen due to **uneven key distribution** or **hot keys**.
- A few consumers may be **overloaded** while others are idle.
- Leads to **high lag** on specific partitions even if overall lag looks normal.



## 6. Processing Bottlenecks

- Slow downstream systems (database, API, file storage).
- Heavy processing logic or large messages.
- Insufficient consumer instances.
- Synchronous calls instead of asynchronous processing.
- Poor batching or inefficient code.
- GC pauses or high resource usage on the consumer.

## 7. When is Consumer Lag Dangerous?

- Lag is **increasing continuously** and not recovering.
- Retention period** may be exceeded (messages deleted before being consumed).
- Business **critical data** is not being processed (e.g. payments, orders).
- Indicates consumer **failure** or **system bottleneck**.
- Causes **real time features** to be delayed.
- May impact **SLAs** and downstream systems.

## 8. How to Reduce Consumer Lag

- Scale **consumer instances**.
- Improve **processing efficiency** (e.g. batching, async calls).
- Fix **slow downstream systems**.
- Redistribute partitions or fix key skew.
- Increase **consumer resources** (CPU, memory).
- Monitor and **alert** on lag growth.

## 9. Useful Commands / Tools

- `kafka-consumer-groups.sh --describe --group <group>`
- Shows current lag for each partition.
- Use monitoring tools (Prometheus, Grafana, Confluent Control Center).
- Check **consumer logs** for errors or slow processing.
- Monitor throughput and processing time.

## 10. Consumer Lag Example

| Partition | Latest Offset | Committed Offset | Lag  | Status   |
|-----------|---------------|------------------|------|----------|
| 0         | 12000         | 11800            | 200  | Normal   |
| 1         | 9500          | 6000             | 3500 | High Lag |
| 2         | 8000          | 8000             | 0    | Healthy  |
| 3         | 15000         | 10000            | 5000 | High Lag |

### Interpretation:

- Partition 1 and 3 have high lag (indicates slow consumer or bottleneck).
- Partition 0 is close to real time.
- Partition 2 is fully caught up.



### Junior Engineer Tip

- Monitor lag per partition, not just total lag.
- Investigate continuous lag growth early.
- Set alerts when lag exceeds a threshold.
- Ensure retention is long enough for recovery.
- Lag is a symptom — find and fix the root cause.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# Kafka Troubleshooting

## 1. Common Kafka Problems

- ❗ Messages are **not being produced** or consumed.
- ❗ High consumer lag.
- ❗ Under-replicated or offline partitions.
- ❗ Broker is down or unstable.
- ❗ High request latency.
- ❗ Disk space issues.
- ❗ Frequent rebalances.
- ❗ Connection or authentication failures.
- ❗ Applications are slow or failing.

## 2. Troubleshooting Workflow



## 3. What to Check at Each Step

- 1 Symptom**
  - What exactly is happening?
  - When did it start?
- 2 Evidence**
  - Error messages, metrics, recent changes
- 3 Broker State**
  - Are all brokers up?
  - Check controller, JVM and disk usage
- 4 Topic / Partition State**
  - Partition count, leader, ISR, replication
- 5 Consumer State**
  - Consumer lag, group status, rebalances
- 6 Logs / Metrics**
  - Broker logs, client logs, monitoring tools
- 7 Hypothesis**
  - Identify the most likely root cause
- 8 Fix**
  - Apply the appropriate fix
- 9 Verification**
  - Confirm normal behaviour and monitor

## 4. Useful Kafka Commands

```

# List all topics
kafka-topics.sh --bootstrap-server <broker:9092> --list

# Describe a topic
kafka-topics.sh --bootstrap-server <broker:9092> --describe --topic <topic>

# Check consumer group lag
kafka-consumer-groups.sh --bootstrap-server <broker:9092> --describe --group <group>

# Check broker status
kafka-broker-api-versions.sh --bootstrap-server <broker:9092>

# Check topic configuration
kafka-configs.sh --bootstrap-server <broker:9092> --describe --topic <topic>

# Get partition and leader info
kafka-topics.sh --bootstrap-server <broker:9092> --describe --topic <topic>
  
```

## 5. Interpreting Key Information

- ✓ **Broker is up**  
Broker responding and healthy
- ✗ **Under-replicated partitions**  
Replicas are not in-sync (risk)
- ⚠ **Offline partitions**  
Partition is not available
- 📊 **High consumer lag**  
Consumers are not keeping up
- 🕒 **High request latency**  
Brokers or clients are slow
- 💾 **High disk usage**  
Risk of running out of space
- 🔄 **Frequent rebalances**  
Consumers joining/leaving often
- 🔗 **Connection failures**  
Network, DNS, firewall or auth issue
- ⚙️ **Authorization errors**

## 6. Example Troubleshooting Flow

Scenario: Messages are not being consumed

- 1 Symptom:** Application is not receiving new messages.
- 2 Evidence:** Check consumer logs and lag.
- 3 Broker State:** Verify all brokers are up.
- 4 Topic / Partition State:** Check partition leaders and ISR.
- 5 Consumer State:** Check consumer group lag and status.
- 6 Logs / Metrics:** Look for errors, rebalances or high latency.
- 7 Hypothesis:** Consumer is slow due to high processing time.
- 8 Fix:** Optimize processing logic or increase consumer instances.
- 9 Verification:** Lag decreases and application processes messages normally.

## 7. Common Root Causes and Fixes

| Problem                     | Possible Cause                          | Fix                                           |
|-----------------------------|-----------------------------------------|-----------------------------------------------|
| High consumer lag           | Slow consumers, insufficient instances  | Scale consumers, optimize processing          |
| Under-replicated partitions | Broker failure, network issue           | Fix broker/network, wait for replicas to sync |
| Offline partitions          | Broker down, disk issue                 | Bring broker back, check disk                 |
| High latency                | High load, insufficient disk I/O        | Scale resources, check disk and network       |
| Connection failures         | Wrong address, firewall, authentication | Verify listeners, network, credentials        |
| Frequent rebalances         | Consumer instability                    | Fix consumer configuration and stability      |

## 8. Verification Checklist

- ✓ Issue is resolved.
- ✓ Consumer lag is normal.
- ✓ No under-replicated or offline partitions.
- ✓ Brokers are healthy.
- ✓ Latency is back to normal.
- ✓ No errors in logs.
- ✓ Monitoring shows stable metrics.



## Junior Engineer Tip

- Always start with the symptom and gather evidence before changing anything.
- Use Kafka commands and monitoring tools to confirm the actual state.
- Check broker, topic/partition and consumer state in order.
- Make one change at a time and verify the result.
- Keep notes of what you checked and what you changed.
- Monitoring and logs are your best friends in production.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# Common Production Failures

## 1. Broker Unavailable



### Symptoms

- Clients cannot connect.
- Metadata requests fail.
- Producing or consuming stops.

### Possible Causes

- Broker is down or crashed.
- Network or firewall issue.
- High resource usage (CPU, memory).
- Disk failure.



### Fix

- Check broker process and logs.
- Verify networking and firewall.
- Check system resources and disk.
- Restart broker if required.

## 2. Leader Unavailable



### Symptoms

- Errors like "Leader not available".
- Partition is not writable.
- Increased producer errors.

### Possible Causes

- Leader broker is down.
- Leader election is in progress.
- Not enough in-sync replicas (ISR).
- Network issue between brokers.



### Fix

- Check broker health.
- Wait for leader election to complete.
- Verify replication and ISR.
- Fix network or disk issues.

## 3. Under-Replicated Partitions



### Symptoms

- ISR is smaller than replication factor.
- Cluster at risk of data loss.
- Reduced fault tolerance.

### Possible Causes

- Broker down or slow.
- Network issues between brokers.
- Disk or I/O problems.
- High produce/replica load.



### Fix

- Bring failed broker back online.
- Check network and disk health.
- Ensure replicas catch up.
- Monitor ISR and replication throttle.

## 4. Disk Pressure



### Symptoms

- High disk usage.
- Brokers become slow or unstable.
- May lead to broker shutdown.

### Possible Causes

- Logs growing due to high traffic.
- Retention settings too high.
- Log cleanup not keeping up.
- Other applications using disk space.



### Fix

- Check disk usage and log directory.
- Adjust retention or cleanup settings.
- Delete old logs if safe.
- Add more disk space or scale storage.

## 5. Authentication Failure



### Symptoms

- Clients cannot authenticate.
- Connection rejected.
- Errors like "SASL authentication failed".

### Possible Causes

- Wrong credentials (username/password).
- Incorrect SASL mechanism.
- Expired certificates (TLS).
- Misconfigured security settings.



### Fix

- Verify credentials and mechanism.
- Check TLS certificates and validity.
- Confirm broker and client security config.
- Check logs for detailed error messages.

## 6. Connection Failure



### Symptoms

- Clients cannot connect to brokers.
- Timeout errors.
- Bootstrap may work but broker connection fails.

### Possible Causes

- Wrong advertised listeners.
- DNS or hostname resolution issue.
- Firewall blocking Kafka ports.
- Incorrect ports or network routing.



### Fix

- Verify bootstrap and advertised listeners.
- Check DNS, firewall and security groups.
- Test connectivity (telnet / nc).
- Ensure correct ports and network setup.

## 7. Rebalance Problems



### Symptoms

- Frequent rebalances.
- Consumers keep joining/leaving.
- Increased processing delay.

### Possible Causes

- Consumer instances are unstable.
- Long processing time (not committing).
- Too many consumers for partitions.
- Network issues or timeouts.



### Fix

- Stabilize consumer instances.
- Optimize processing time.
- Adjust max.poll.interval.ms and timeouts.
- Ensure proper partition to consumer ratio.

## 8. Message Processing Failures



### Symptoms

- Messages are not processed.
- Repeated retries.
- Messages may go to a dead letter topic.

### Possible Causes

- Application errors (code, validation).
- External dependency failures (DB, API).
- Malformed or unexpected message data.
- Resource issues (CPU, memory).



### Fix

- Check application logs for errors.
- Verify message format and data.
- Fix code or dependency issues.
- Use retry and dead letter topic (DLT).

## 9. Growing Consumer Lag



### Symptoms

- Lag is continuously increasing.
- Consumers are not keeping up.
- Delayed message processing.

### Possible Causes

- Slow consumer processing logic.
- Insufficient consumer instances.
- Partition imbalance.
- External system is slow (DB, API).



### Fix

- Identify slow consumers and optimize.
- Increase consumer instances.
- Rebalance partitions if needed.
- Fix downstream dependencies.

## 10. Quick Troubleshooting Checklist

- 1 Check symptoms and error messages.
- 2 Verify broker health and cluster state.
- 3 Check topic and partition state (leader, ISR, offsets).
- 4 Check consumer state and lag.
- 5 Review broker, client and application logs.
- 6 Identify the most likely cause.
- 7 Apply the fix.
- 8 Monitor to confirm the issue is resolved.

## 11. Common Kafka Error Messages

| Error Message                   | Meaning                           |
|---------------------------------|-----------------------------------|
| Leader not available            | Partition leader is not available |
| Not enough replicas             | Not enough in-sync replicas (ISR) |
| SASL authentication failed      | Invalid credentials or mechanism  |
| Timeout while fetching metadata | Cannot connect to broker          |
| Broker not available            | Broker is down or unreachable     |
| Not authorized to access topic  | ACL or permission issue           |
| Corrupt record                  | Message format is invalid         |



### Junior Engineer Tip

- Always check logs first.
- Most Kafka issues are visible in broker, client or application logs.
- Monitor key metrics (URP, lag, disk, request latency).
- Fix the root cause, not just the symptom.
- Set alerts to detect issues early.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

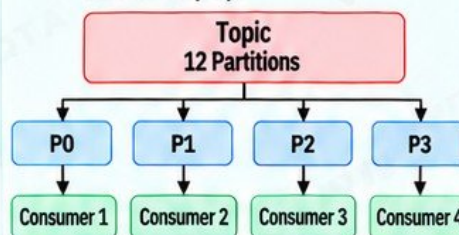
# Performance & Scaling

## 1. What Affects Kafka Performance?

- Number of **partitions** and parallelism.
- Producer **batching** and **compression**.
- **Throughput** versus **latency** requirements.
- **Broker capacity** (CPU, memory, disk, network).
- **Consumer processing** speed.
- Disk and network **I/O bottlenecks**.
- **Replication factor** and ISR health.
- **Message size** and serialization format.

## 2. Partitions and Parallelism

- Each **partition** is a **sequential** log.
- More **partitions** allow higher **parallelism** (more consumers).
- Partitions can be processed in **parallel** across consumers.
- However, **too many partitions** can cause:
  - Higher memory usage.
  - Increased metadata overhead.
  - Longer rebalances.
  - Diminishing returns if consumers cannot keep up.



## 3. Producer Batching and Compression

- Producer **batches** multiple records before sending to reduce overhead.
- Larger batches improve **throughput** but increase **latency**.
- **Compression** reduces message size and network/disk usage.
- Common compression types:
  - **gzip** (good compression, more CPU)
  - **snappy** (balanced, commonly used)
  - **lz4** (fast, lower CPU usage)
  - **zstd** (high compression, good performance)

### Producer Settings (examples)

```

batch.size=32768      # 32 KB
linger.ms=5           # wait up to 5ms
compression.type=snappy # or lz4 / zstd
  
```

## 4. Throughput vs Latency

- **Throughput**: number of messages processed per second.
- **Latency**: time taken for a message to be written and consumed.
- Larger **batches** and **compression** increase throughput but add **latency**.
- **Smaller batches** reduce latency but may lower throughput.
- Choose settings based on your use case (real-time vs bulk processing).

### Higher Throughput (larger batches)

- More messages/sec
- Higher latency
- Better resource usage

### Lower Latency (smaller batches)

- Faster delivery
- Lower latency
- May reduce throughput

## 5. Broker Capacity and Scaling

- Brokers need enough **CPU**, **memory**, **disk** and **network** to handle load.
- Monitor key metrics:
  - CPU usage
  - Memory usage (JVM heap)
  - Disk I/O and disk space
  - Network in/out
  - Request queue size
- Scale by:
  - Adding more brokers.
  - Distributing partitions evenly.
  - Ensuring ISR is healthy.
  - Avoid hotspots on a single broker.



## 6. Disk and Network Bottlenecks



### Disk Bottlenecks

- High disk I/O from writes and reads.
- Slow disks can increase latency and reduce throughput.
- Monitor disk usage and I/O wait.
- Use fast storage (SSD) for better performance.



### Network Bottlenecks

- Network bandwidth can limit throughput.
- High replication traffic increases network usage.
- Monitor network in/out and packet drops.
- Use proper network configuration and avoid cross-region latency.

## 7. Why "Add More Partitions" is Not Always Correct



More partitions increase parallelism, but:

- Does not help if consumers cannot keep up.
- Can cause longer rebalances.
- Increases memory and metadata overhead.
- May create small partitions with low throughput.
- Does not fix disk, network or consumer bottlenecks.
- Can make operational management harder.

Add partitions only when you need more parallelism and your consumers and infrastructure can handle it.

## 8. Performance Best Practices

- ✓ Use appropriate partition count.
- ✓ Tune producer batching and compression.
- ✓ Monitor and size brokers correctly.
- ✓ Keep ISR healthy.
- ✓ Use fast storage (SSD).
- ✓ Monitor disk and network usage.
- ✓ Balance load across brokers.
- ✓ Optimize consumer processing.
- ✓ Test and measure (throughput, latency).
- ✓ Scale based on real metrics, not guesswork.

## 9. Key Takeaways

- 1 Performance depends on producers, consumers, brokers and infrastructure.
- 2 Batching and compression improve throughput.
- 3 More partitions help parallelism but are not a magic fix.
- 4 Monitor disk, network, CPU and memory.
- 5 Tune based on your requirements (throughput vs latency).
- 6 Scale thoughtfully and test changes.



### Junior Engineer Tip

- Always identify the real bottleneck before changing partition count.
- Use metrics to make decisions.
- Test changes in a non-production environment first.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

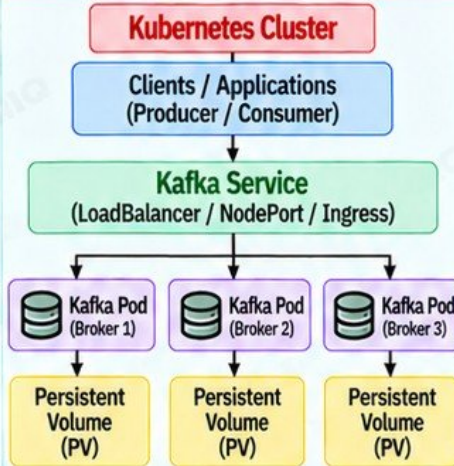
# Kafka on Kubernetes & Cloud

## 1. Kafka as a Stateful Workload

- Kafka is **stateful** because it stores all messages **on disk**.
- Data must **persist across** pod restarts.
- Each **broker** has a **unique identity** (broker.id) and its own storage.
- Cannot be treated like a stateless** application.
- Requires **careful storage**, networking and scaling considerations.

**!** Unlike stateless apps, you cannot simply delete and recreate Kafka pods without considering data and cluster state.

## 2. Kubernetes Architecture



## 3. Persistent Storage

-  Kafka stores logs on disk, so use **Persistent Volumes (PV)**.
- Use **fast** and **reliable** storage (SSD recommended).
- Each broker should have its own PV.
- Configure **storage class** with appropriate **IOPS** and throughput.
- Ensure **enough disk space** for **retention** and **future growth**.

**!** **Risks**

- Using slow storage causes high latency.
- Running out of disk space stops writes.
- Deleting PV can lead to data loss.

## 4. StatefulSets and Operators

- Use **StatefulSets** to manage Kafka brokers.
- Each pod gets a **stable network identity** and persistent volume.
- Operators** (e.g. Strimzi, Confluent Operator) simplify deployment, scaling and management.
- Operators handle configuration, rolling updates and monitoring.

### Popular Operators



## 5. Networking Considerations

- Each broker needs to be reachable by clients and other brokers.
- Use **headless service** for internal broker communication.
- Configure **advertised.listeners** with the correct DNS or external address.
- For **external access**, use LoadBalancer, **NodePort** or **Ingress** (with TLS).
- Ensure proper DNS, firewall rules and security groups.

### Example Listeners

```
listeners=PLAINTEXT://:9092,
EXTERNAL://:9094
advertised.listeners=PLAINTEXT://kafka-0.kafka:9092,
EXTERNAL://kafka.example.com:9094
```

## 6. Handling Disruptions

-  Pods can be rescheduled, so Kafka must handle restarts gracefully.
- Use **PodDisruptionBudgets (PDB)** to prevent multiple brokers from going down.
- Perform **rolling updates**, not delete and recreate.
- Ensure sufficient replication factor and **min.insync.replicas**.
- Test node failures and upgrades in non-production environments.



### Best Practices

- Use PDB.
- Do rolling upgrades.
- Maintain ISR.
- Verify cluster health after upgrades.

## 7. Availability Zones and High Availability

- Deploy brokers across multiple availability zones (AZs).
- Ensure **replicas** are distributed across AZs.
- Use a **replication factor** of at least 3 in production.
- Configure rack awareness to keep replicas in different AZs.
- This protects against zone failures and improves availability.



## 8. Managed Kafka Services (Cloud)

- Managed services remove the operational burden of running Kafka.
- Examples:
  -  Amazon MSK (AWS)
  -  Confluent Cloud
  -  Azure Event Hubs / Confluent on Azure
  -  Google Cloud Managed Kafka (Confluent on GCP)

### Benefits

-  Automatic scaling.
-  Managed upgrades.
-  High availability by default.
-  Reduced operational risk.

## 9. Why Kafka is Different from a Stateless Application

- Kafka stores persistent data on disk.
- Brokers have unique identities and state.
- Requires careful storage, networking and scaling.
- Cannot use auto-scaling like a stateless app.
- Data consistency and replication must be maintained.
- Upgrades and failures must be handled carefully to avoid data loss.
- Operations involve more planning, monitoring and risk management.



### Junior Engineer Tip

- Always treat Kafka as a stateful system.
- Plan for storage, networking and AZs from the beginning.
- Test failure scenarios.
- Use managed services if you don't want to handle the operational complexity.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

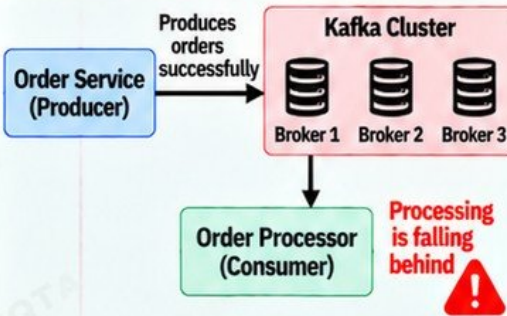


@veriqta

# Production Incident Lab

**Scenario: Orders are being produced successfully, but processing is falling behind.**

## 1. Environment Overview



- Orders are being produced successfully.
- Messages are stored in Kafka.
- The consumer group is not keeping up.
- You need to investigate and fix the issue safely.

## 2. Observe the Symptoms

- Producers are working and sending messages.
- Consumer lag is increasing.
- Orders are not being processed in real time.
- No errors on the producer side.
- System appears healthy but processing is delayed.



### Questions to Answer

- Is the consumer lagging?
- Which partitions are affected?
- Are consumers healthy?
- Is there a processing bottleneck?
- What is the safest fix?

## 3. Check Consumer Lag

```
# Check consumer group lag
kafka-consumer-groups.sh \
  --bootstrap-server <broker>:9092 \
  describe --group order-processor
```

| Topic  | Partition | Current Offset | Log End Offset | Lag   |
|--------|-----------|----------------|----------------|-------|
| orders | 0         | 12000          | 20000          | 8000  |
| orders | 1         | 15000          | 25000          | 10000 |
| orders | 2         | 18000          | 30000          | 12000 |



High lag means the consumer is not keeping up with new messages on those partitions.

## 4. Check Partition Distribution

```
# Check partition details
kafka-topics.sh --bootstrap-server
<broker>:9092 --describe --topic orders
```

| Partition | Leader | Replicas | ISR   |
|-----------|--------|----------|-------|
| 0         | 1      | 1,2,3    | 1,2,3 |
| 1         | 2      | 2,3,1    | 2,3,1 |
| 2         | 3      | 3,1,2    | 3,1,2 |

- Ensure partitions are evenly distributed across brokers.
- Check for any offline or under-replicated partitions.
- Verify ISR includes all replicas.

## 5. Check Consumer Health

```
# Check consumer group details
kafka-consumer-groups.sh --bootstrap-server
<broker>:9092 --describe --group order-processor
```

- Check the number of active consumers.
- Verify each consumer is assigned partitions.
- Look for consumers that are down or constantly rebalancing.
- Check consumer logs for errors, timeouts, or retries.



### Common Problems

- Not enough consumer instances.
- Consumers are frequently rebalancing.
- Consumers are processing too slowly.
- Consumer crashes or OOM errors.

## 6. Check Logs and Metrics



### Logs to Check

- Consumer application logs (errors, retries, exceptions).
- Kafka broker logs (network, disk, ISR).
- Rebalance events in consumer logs.



### Key Metrics

- Consumer lag (per partition).
- Consumer processing time.
- CPU and memory usage.
- Broker request latency.
- Disk usage and network I/O.
- JVM health (GC, heap usage).

## 7. Identify the Bottleneck

- Compare producer rate vs consumer rate.
- Check if a specific partition has high lag.
- Identify slow processing logic in the consumer.
- Look for external dependencies (database, API).
- Check consumer resource usage (CPU, memory).
- Determine if more consumers are needed.



### Example Findings

- Partition 2 has the highest lag.
- Consumer CPU is at 100%.
- Processing logic is slow due to an external API.
- Only 2 consumer instances for 6 partitions.

## 8. Apply a Safe Corrective Action

- Scale consumer instances.
- Optimize processing logic (reduce API calls, use batching).
- Increase consumer resources (CPU, memory).
- Fix failing external dependencies.
- Rebalance partitions if necessary.
- Tune consumer configuration (max.poll.records, timeouts).



### Example Action

Scale consumer deployment from 2 to 6 replicas to handle the load and reduce lag.

### Command Example (Kubernetes)

```
kubectl scale deployment order-processor \
  --replicas=6
```

## 9. Verify Recovery

- Check consumer lag again (should decrease to near zero).
- Ensure consumers are stable (no frequent rebalances).
- Monitor processing rate and system metrics.
- Confirm orders are being processed in real time.
- Keep monitoring for at least 30 minutes to ensure stability.



### Expected Result

- Consumer lag is low or zero.
- Processing is back to real time.
- Consumers are healthy and stable.
- No new errors in logs.
- System remains stable under load.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta